Skip List Implementation and Experimental Analysis

Individual take-home challenge · C or C++ · Maximum 3 points
The maximum exam score is 32. Upload your project to the course WeBeep area by the deadline announced there. Your report must be a PDF of at most two pages.

1 Assignment and scope

Implement a randomized skip list representing a set of distinct signed 64-bit integer keys. Develop four kernels: **search, insert, ordered union and split**. Test their correctness and use Google Benchmark to investigate how their cost scales with input size. The challenge assesses algorithm design, ownership of nodes, reproducible experiments and your interpretation of the evidence.

Section 10 defines cumulative grading and partial credit. The complete requirements in Sections 4, 5 and 7 apply to the 2 and 3 point levels; Section 6 adds the investigation required for 3 points.

Use C17 or C++17, or a later documented standard. A C implementation must be linked to a C++ Google Benchmark driver. Implement the data structure yourself; standard utilities are allowed, but a library container or ready-made skip list must not implement the required operations. Reference containers are allowed in tests. Deletion, concurrency, rank/select, persistence and generic payloads are outside the required scope.

2 Required operation contracts

Kernel	Required behaviour
search(S, k)	Return whether k belongs to S. Leave S unchanged.
insert(S, k)	Insert k only if absent and report whether insertion occurred. A duplicate changes neither the set nor existing node heights.
ordered_union(A, B)	Return all keys from A and B. Inputs must be distinct; when both are nonempty, every key in A must be strictly smaller than every key in B. Empty inputs are valid. Consume A and B.
split(S, x)	Return L = {k in S : k ≤ x} and R = {k in S : k > x}. The pivot need not be present. Consume S.

These are the union and split semantics used in the lecture [1]. Use a name such as `ordered_union`, since `union` is a C/C++ keyword. Handling overlapping or incorrectly ordered union inputs is outside the contract.

Ownership: consumed input objects must remain valid, empty and reusable. Outputs must own their data nodes independently and support further operations. Each node must eventually be freed exactly once. Document your interface and ownership rules.

3 Randomized representation

Use independently sampled geometric node heights with promotion probability p = 1/2 for the main experiments. Make seeds configurable and reproducible; keep workload generation separate from height generation. Explain your sentinel and height representation. If heights are capped, state and justify the cap for the tested size range.

Resources

[1] Course lecture on Randomized Data Structures, especially slide 6 and the skip-list section.

[2] Google Benchmark and its official user guide.

[3] W. Pugh, *A Skip List Cookbook*, revised June 1990. University of Maryland technical report (supplementary).

Algorithm design and validation

4 Targets for the 2 and 3 point levels

For these levels, your primary implementations of ordered union and split must **rewire existing forward links and transfer existing data nodes**. Preserve each node's address, key and height. Rebuilding through reinsertion or copying, or assigning fresh heights, is not a structural implementation. A constant number of new list headers is allowed.

Operation	Expected time target
search and insert on n keys	O(log(n + 2)) per operation
ordered union of n and m keys	O(log(n + m + 2))
split of n keys	O(log(n + 2))

The expectation is over random heights. Give a short argument for these targets and expected O(n) storage under your representation and height policy; a full high-probability proof is not required. With a fixed height cap, acknowledge the supported range. See [3] for supplementary background.

Include all work performed by the operation in its analysis. Maintaining metadata or repairing sentinel links may hide a full traversal. Constant-time `size()` after split is not required; a diagnostic traversal may be used outside the kernel. The benchmark may know input cardinalities from its generator.

A correct implementation with slower, rebuilding versions of union or split remains eligible for **1 point**, provided the report accurately states its costs.

5 Automated correctness checks

Provide an ordered traversal and an invariant checker. Check increasing keys at every level, all keys at the bottom level, upper levels as subsequences of lower levels, valid heights and links, and absence of cycles. Compare contents and operation results with an independent reference, such as `std::set` or a sorted vector.

- Cover empty and singleton inputs, duplicates, present and absent searches, and signed 64-bit boundary keys without sentinel collisions.
- Split before all keys, after all keys and exactly at a stored key. Union with empty operands, very unequal input sizes and different heights.
- Verify that ordered_union(split(S, x)) restores the original set. Modify both split outputs independently and verify that neither affects the other. Check reuse of consumed inputs; at the structural levels, also check preservation of node addresses and heights.
- Run at least 1,000 mixed operations for each of three documented seeds, checking results against the reference. Include repeated split/union sequences. Run a memory checker, such as sanitizers or Valgrind, and state the outcome.

6 One investigation for full credit

For 3 points, compare your structural implementation with **one rebuilding baseline** for either ordered union or split. The baseline must implement the same public contract by traversing the input, creating fresh outputs using ordinary insertions, and releasing the consumed nodes. Predict its cost, compare both methods on the same workloads, and explain the observed difference.

In separate runs, instrument all four structural kernels with a precisely defined logical-work count, such as key comparisons and link traversals/updates. For the chosen comparison, also count data-node allocations. Use this evidence to interpret timing trends. Disable instrumentation during timing runs. No particular speedup is required.

Experimental protocol

7 Google Benchmark requirements

Use the official Google Benchmark library [2] for all four kernels. Keep correctness checks and instrumentation separate from optimized timing runs. Record the library version, compiler and flags, operating system and processor. Run the kernels on a single thread.

Sizes and randomness. Use at least six geometrically spaced sizes spanning a substantial range; for example $N = 2^{10}, 2^{12}, 2^{14}, 2^{16}, 2^{18}, 2^{20}$. A smaller range is acceptable if memory or runtime limits are explained. For each workload and size, use at least three independent height seeds and three timing repetitions per seed. Repetitions of one fixed structure alone do not sample the randomness of the data structure.

Kernel	Main workload and meaning of N
search	N stored keys; pre-generate queries with 50% hits and 50% misses, spread across the key range. State how queries are sampled.
insert	N keys before insertion; every timed insertion must use an absent key. Keep the starting size controlled across trials.
ordered union	N = |A| + |B|; use disjoint ordered ranges and |A| = |B| = N/2. Build inputs with independently generated heights.
split	N keys before the operation; choose a pivot giving approximately equal output sizes. Determine the pivot before timing.

Different size ratios, pivot positions or query mixes are separate benchmark families. Boundary cases belong in correctness tests; additional performance families are optional.

Measure the kernel and control its state

Exclude input construction, workload generation, validation, reference-container work, output, and restoration from the timed region. Include work intrinsic to the operation: allocation and height generation during insert, header handling, and all metadata updates. The rebuilding baseline includes its own allocations and disposal of consumed nodes.

Restore valid, equivalent inputs before the next measurement. Repeated insertion of an existing key, unbounded growth of a list, or repeated union of already consumed inputs measures a different workload. Untimed restoration may use helpers, including a private erase helper; implementing public deletion is not required.

Use appropriate timing controls and optimization barriers, including `PauseTiming/ResumeTiming`, `DoNotOptimize` and `ClobberMemory` where appropriate [2]. Avoid rebuilding a large input for every tiny operation with unlimited calibration. Bounded iterations, reversible restoration and fixed-size batches are acceptable; document your choice.

Estimate and interpret growth

Set `state.SetComplexityN(N)` and enable `Complexity()` or `Complexity(model)` for each kernel family. Report the fitted model, coefficient and normalized RMS error, and distinguish an automatically selected model from one supplied by you. A supplied logarithmic model is a hypothesis being fitted. Preserve raw JSON outputs [2].

If batching, keep the number B of timed operations per iteration fixed across N. For insert, a documented size interval [N, N+B] is acceptable. Normalize plotted time and fitted coefficients by B. `SetItemsProcessed` alone does not normalize the times used in the complexity fit. Never report an N-operation batch as one operation.

A fit is not a proof of asymptotic complexity. Compare theory and timing evidence; for full credit, also use logical-work counts. Discuss overhead, cache effects, random variability or a limited size range. An unexpected model or noisy logarithmic trend is acceptable when analysed honestly.

Report submission and assessment

8 Two-page report

Submit `report.pdf` in English: **at most two A4 pages** including figures, tables and references, with at least 10-point body text and 1.5 cm margins. Include your name and student ID. No cover page or appendix.

Describe representation, ownership and expected costs; summarize methods and correctness checks; show scaling results for all four kernels; and discuss conclusions and a material limitation. Full credit also requires the baseline comparison and logical-work evidence. A multipanel figure and compact complexity table are suitable. Keep full commands, seeds and raw results in the archive.

9 Submission on WeBeep

Upload one ZIP or TAR.GZ archive to the **course WeBeep submission area** by the announced deadline. Include sources, Makefile or CMake configuration, tests, Google Benchmark code, `report.pdf`, raw results, and figure-generation scripts or precise plotting instructions.

The README must give exact build/test/benchmark commands, dependency versions, seeds, memory-check results, limitations, source/tool acknowledgements and a declaration of individual work. Exclude binaries, build directories and downloaded dependencies.

10 How marks are awarded

The following levels are cumulative. The highest fully demonstrated level determines the mark; the maximum is 3 points.

Mark	Evidence required
1 point Functional solution	All four kernels work on ordinary and essential boundary cases. The project builds, includes automated checks and runnable Google Benchmark families, and provides a compliant report with initial scaling evidence. Clearly identified rebuilding union/split is acceptable.
2 points Sound algorithms	All 1-point requirements, plus structural union/split, safe ownership, preserved nodes and heights, expected-cost arguments, and the full correctness and experimental protocol in Sections 4, 5 and 7. Results are reproducible and correctly interpreted.
3 points Convincing analysis	All 2-point requirements, plus the controlled comparison and logical-work evidence in Section 6. The report connects algorithmic choices, expected costs and observations, explaining limitations or disagreement between measurements and predictions.

Minor defects are assessed in proportion to their impact; work without a substantively working solution receives 0 points. Extra features, absolute speed, a favourable fitted label or faster hardware do not by themselves earn credit.

11 Individual work and academic integrity

This project must be developed **individually**. Do not co-author or share solution code, results or report text. References and development tools, including AI assistants, are allowed; disclose sources and assistance in the README. Attribution does not make submitting a pre-existing skip-list implementation acceptable. Do not submit copied code as your own. Verify and understand every part of your submission.

I will use code-similarity analysis and other anti-cheating tools to check whether submitted code has been copied. If I have doubts about originality or authorship, I may call the student for an oral discussion of the project. This may include explaining an algorithm or result, tracing pointer changes, or making a small code modification.